Ministerul Educaţiei şi Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică şi Microelectronică
Departamentul Ingineria Software şi Automatică
Program: Software Engineering

Report on laboratory work no. 4

Course: Operating Systems

Topic: The xv6 Shell from the Outside

Track B: build your own small OS, xv6

Done by:
st. gr. FAF-241,
Serghei Barladean

Checked by:
Patricia Reitman

Chişinău – 2026

Contents

Purpose of the Work **3**
Part 1. Fix sleep for the Current xv6 ..... **3**
1.1. Work Done 3
Part 2. bash versus the xv6 Shell **4**
2.1. Comparison Table 4
2.2. Figures 5
2.3. Observations 7
Part 3. Who Expands *? Printing argv **7**
3.1. Work Done 7
3.2. Observations 8
Part 4. Read the Shell **9**
4.1. Work Done 9
4.2. Observations 10
Conclusions **11**

Purpose of the Work

The purpose of this laboratory work is to use the xv6 shell side by side with bash and to find out, by experiment, what a shell must do and what bash adds on top of it. A small program that prints its arguments shows who expands * and who removes the quotes. At the beginning the sleep program from Laboratory Work 2 is checked on the current xv6.

The work was done on Ubuntu 24.04 (aarch64) in a Multipass virtual machine on macOS. xv6 (RISC-V) was built in ~/xv6-riscv and run in the QEMU emulator; the Linux part was run in bash. The figures are taken from the recorded terminal sessions.

Part 1. Fix sleep for the Current xv6

1.1. Work Done

In August 2025 the xv6 authors renamed the system call `sleep()` to `pause()`. The name used by the tree was checked with:

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

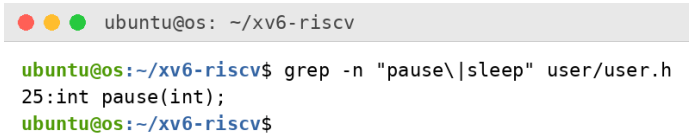


Figure 1.1 – The grep result in user.h

The only match is `25:int pause(int);`: the tree declares pause and no sleep, so the call in user/sleep.c must be pause(). The updated program and the line `$U/_sleep\` in the UPROGS list of the Makefile (a tab at the start, nothing after the backslash) are shown below:



Figure 1.2 – The updated sleep.c and the UPROGS lines in the Makefile

After make qemu the program was tested inside xv6. With the argument 20 it waited for 20 clock ticks and returned to the prompt; without an argument it printed the usage message:

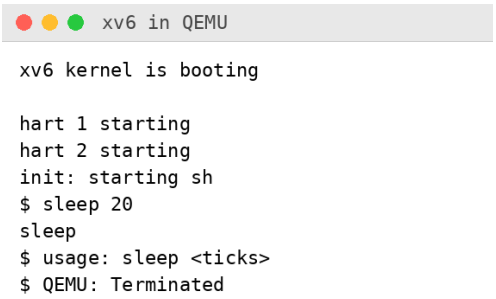


Figure 1.3 – sleep running in xv6, with and without an argument

The second line sleep in the figure was typed while sleep 20 was still running; the usage message appears after it finished.

Part 2. bash versus the xv6 Shell

2.1. Comparison Table

Every command was run once in bash, in the empty test folder ~/shtest that contains a file named README, and once at the xv6 prompt:

Command	Tests	bash	xv6	Why
`echo hello > f ; cat < f`	redirection	hello	hello	Both shells implement < and > and the sequence operator ;.
`ls | grep READ`	pipe	README	README 2 2 2441	Both support pipes. xv6 ls prints the type, inode number and size of each file.
`(echo a; echo b) | wc`	grouping, list	2 2 4 (padded with spaces)	2 2 4	Both support parentheses and pipes. GNU wc pads the numbers, the xv6 wc does not.
`echo one ; echo two`	sequence	one two	one two	Both run the commands one after another.
`echo bg &`	background	[1] 52386 bg [1]+ Done echo bg	bg	Both run the command in the background. Only bash does job control and prints job numbers and status.
`ls *`	globbing	README f	ls: cannot open *	bash expands * into file names; xv6 passes the literal * to ls, which has no file of that name.
`echo $HOME`	variables	/home/ubuntu	$HOME	xv6 shell has no variables; the text is printed as it is.
`echo "a b"`	quotes	a b	"a b"	bash removes the quotes; xv6 does not know quotes: echo receives the two words "a and b" with the quote characters.
`mkdir d ; cd d ; ls ..`	PATH	README d f (and the shell is now in d)	exec cd failed (listing of the root directory)	In xv6 cd is recognised only when it is the whole line, so inside a list it is run as a program and fails. The ls .. then ran in the old directory.
arrow up, Tab, Ctrl+R	history, completion	arrow up repeated the previous command; Tab on "ec" listed ec2metadata and echo	arrow up: exec ^[[A failed; Tab: exec ec failed	bash has line editing and history (readline); xv6 reads raw characters, so the keys are passed to the shell as text. Ctrl+R was not run.

2.2. Figures

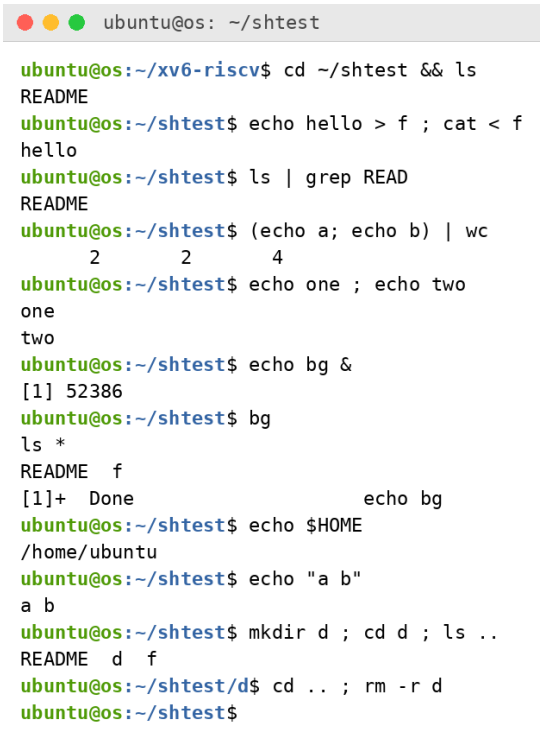


Figure 2.1 – The commands run in bash

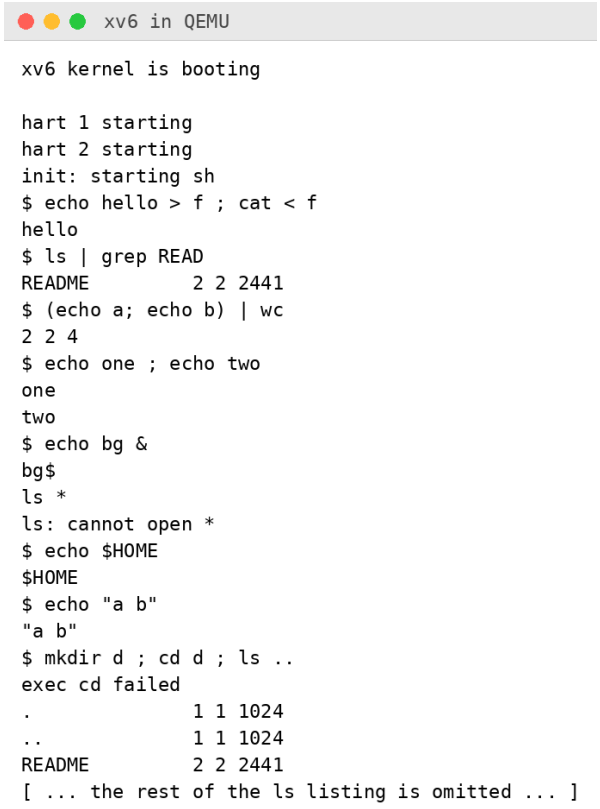


Figure 2.2 – The same commands run in xv6 (the long ls listing is shortened)

In xv6 the program `ls` was found after `cd d` only when it was started by a path. The commands and the result were:

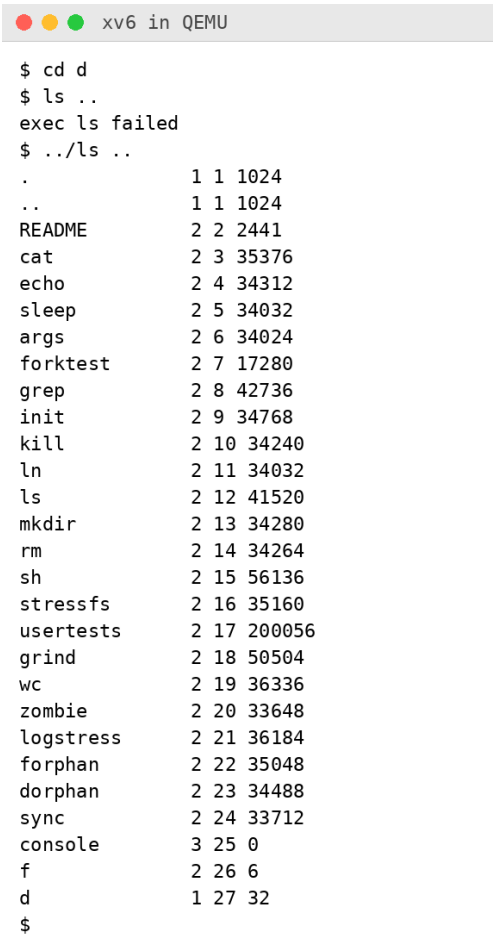


Figure 2.3 – ls failing after cd d, and the working alternative

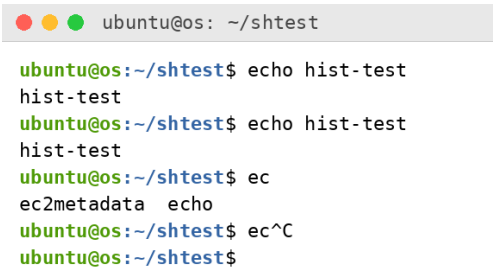


Figure 2.4 – Arrow up and Tab in bash

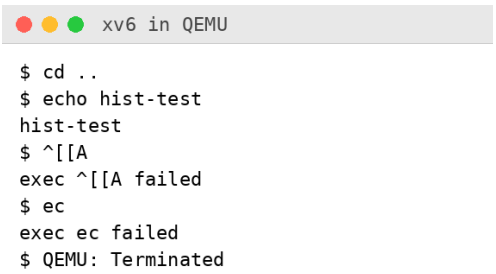


Figure 2.5 – Arrow up and Tab in xv6

2.3. Observations

1. Both shells can run programs, redirect input and output (<, >), build pipes (|), run commands one after another (;), group commands with parentheses and run a command in the background (&). Only bash has globbing (*), variables ($HOME), quote processing, a PATH to search for programs, job control messages, command history, completion and line editing, and it recognises cd inside a list of commands.
2. `ls` stops working after `cd d` because the xv6 shell has no PATH: it runs exec with the name as typed, and exec looks for that name in the current directory. The xv6 programs live in the root directory of the xv6 file system, so after cd d there is no file called ls there ("exec ls failed"). It works when the program is started by a path, `../ls ..` (or `/ls ..`), as in Figure 2.3.
3. In daily work I would miss command history and completion most: without the arrow keys every command has to be typed again, and a typing mistake means retyping the whole line. The easiest feature to add would be a simple search for programs: when exec of a name fails, the shell can try the same name in the root directory (a few lines in the function that runs a command).

Part 3. Who Expands *? Printing argv

3.1. Work Done

The program user/args.c prints the number of arguments and each argument in square brackets, so that spaces and quotes are visible. The line `$U/_args\` was added to UPROGS:

```
ubuntu@os: ~/xv6-riscv
ubuntu@os:~/xv6-riscv$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
ubuntu@os:~/xv6-riscv$ grep -n "_args" Makefile
136:    $U/_args\
ubuntu@os:~/xv6-riscv$
```

Figure 3.1 – args.c and the UPROGS line

The same program for Linux, args_linux.c, uses `<stdio.h>` and `return 0` and was compiled with `gcc -o args args_linux.c`. Both programs were run with the same four lines, in bash in a folder that contains a.c and b.c, and in xv6:

```
args one              two         three
args *.c "x y"
args $HOME
args 'single quoted'
```

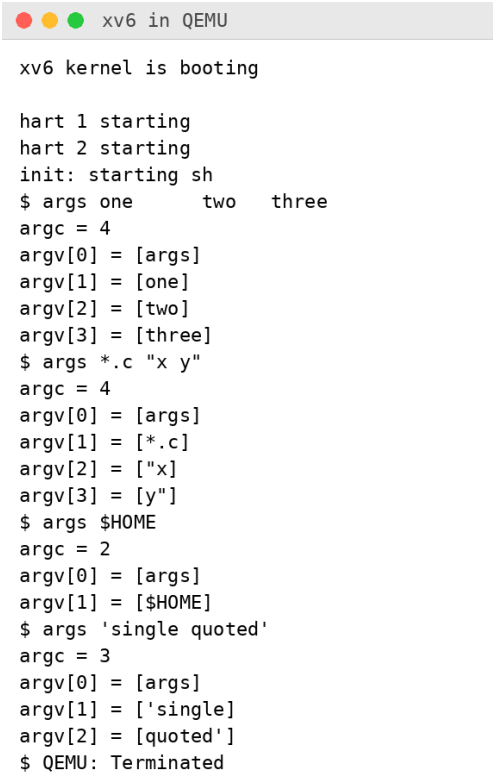


Figure 3.2 – args output in xv6

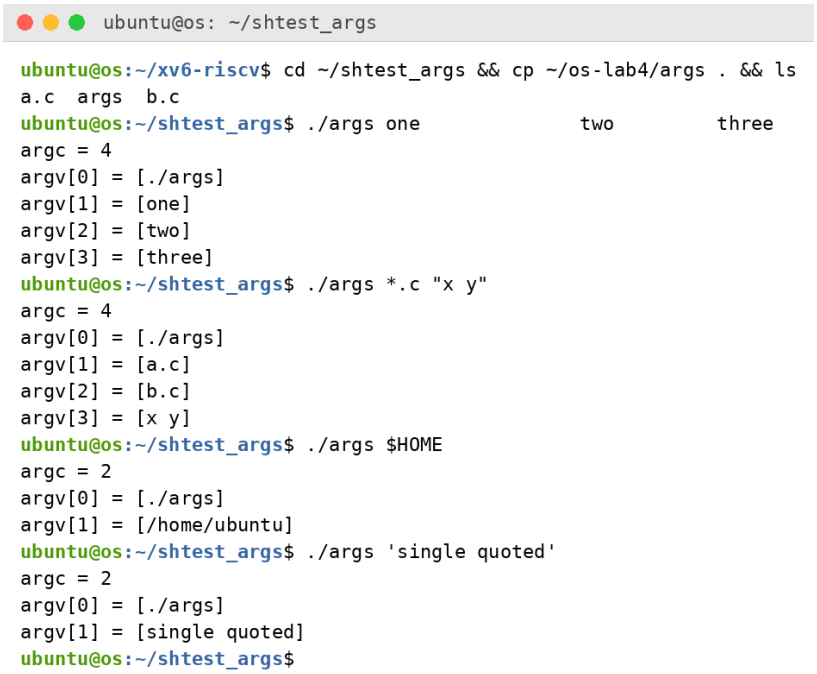


Figure 3.3 – args_linux output in bash

3.2. Observations

1. `args *.c "x y"`: in bash the output was argc = 4 with argv = [./args], [a.c], [b.c], [x y]. In xv6 the output was argc = 4 with argv = [args], [*.c], ["x], [y"]. Both programs received 4 arguments, but not the same ones: bash replaced *.c by the two file names and kept "x y"

as one argument, xv6 passed *.c unchanged and split "x y" into two words that still contain the quote characters.

2. The shell does both. It expands *.c into file names and removes the quotes before it calls exec; the kernel only copies the argument list to the new program, and the program just prints what it received. This is also why the same program gives different results in the two shells.
3. The extra spaces in one two three disappeared in both shells: the shell uses runs of spaces only as separators between words, so the program received the three arguments one, two and three (argc = 4, counting the program name).

Part 4. Read the Shell

4.1. Work Done

The file user/sh.c has 499 lines. The kinds of commands, the places where each kind is handled and the handling of cd were found with grep:

```
wc -l user/sh.c
grep -n "#define" user/sh.c
grep -n "case " user/sh.c
grep -n "cd" user/sh.c | head
```

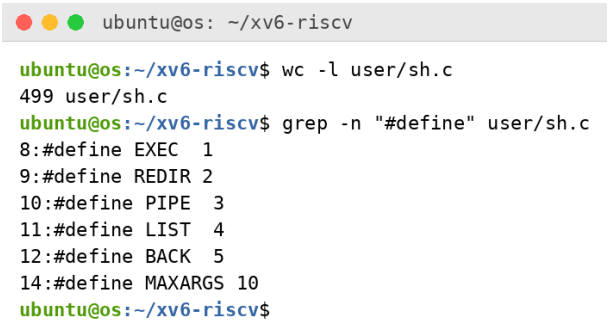


Figure 4.1 – The command kinds defined in sh.c

```
ubuntu@os: ~/xv6-riscv
ubuntu@os:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
ubuntu@os:~/xv6-riscv$
```

Figure 4.2 – Where each kind is run

```
ubuntu@os: ~/xv6-riscv
ubuntu@os:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
ubuntu@os:~/xv6-riscv$ grep -n -B7 -A4 "cannot cd" user/sh.c
163-      cmd++;
164-    if (*cmd == '\n') // is a blank command
165-      continue;
166-    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
167-      // Chdir must be called by the parent, not the child.
168-      cmd[strlen(cmd) - 1] = 0; // chop \n
169-      if (chdir(cmd + 3) < 0)
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
171-    } else {
172-      if (fork1() == 0)
173-        runcmd(parsecmd(cmd));
174-      wait(0);
ubuntu@os:~/xv6-riscv$
```

Figure 4.3 – The code that handles cd

A search for the characters * and $ in the source gave no result:

```
ubuntu@os: ~/xv6-riscv
ubuntu@os:~/xv6-riscv$ grep -n "'\*'\|'\$'" user/sh.c || echo "(no matches)"
(no matches)
ubuntu@os:~/xv6-riscv$
```

*Figure 4.4 – Search for code that handles * or $ (no matches)*

4.2. Observations

1. The xv6 shell knows five kinds of commands (lines 8–12 of sh.c): EXEC, a simple command, a program with its arguments (plain words); REDIR, redirection, produced by < , > and >> ; PIPE, produced by | ; LIST, two commands one after another, produced by ; (parentheses are used to group them); BACK, a command in the background, produced by & . The case labels at lines 75–125 and 469–493 handle these five kinds.

2. cd is handled in the main loop of the shell (lines 166–170), before a child process is created: if the line starts with "cd ", the shell calls chdir itself in the parent process. It is not run as a separate program because the working directory belongs to the process: a child that called chdir would change only its own directory and then exit, and the shell would stay in the old one. The same check also explains "exec cd failed" in Part 2: it works only when the whole line starts with cd; in mkdir d ; cd d ; ls .. the part cd d is run by a child as a program, and no such program exists.
3. The search for the characters * and $ in sh.c found nothing ("no matches"). The xv6 shell therefore has no code for globbing or variables, which explains Part 2 and Part 3: ls * passed a literal * to ls, echo $HOME printed $HOME, and args received *.c and $HOME unchanged. The quote characters also reached the programs unchanged, so the shell does no quote processing either.

Conclusions

The xv6 shell is a very small program that does only what a shell must do: start programs, redirect their input and output, connect them with pipes, run lists of commands and background commands, and handle cd itself. Everything else that was seen in bash, globbing, variables, quotes, PATH, job control and history, is an addition made by bash.

The args program showed the division of work: the shell builds the argument list (expands * and removes the quotes in bash, does neither in xv6), the kernel passes this list to the program through exec, and the program only prints what it received.

The sleep program from Laboratory Work 2 was fixed for the current xv6, where the system call sleep() is called pause(), and it was tested inside xv6.